

Lab 2 Report: Assembler

Amaya Francom

A02395949

Jackson Gardner

A02340229

Vanessa Bingham

A02274022

Objective

The first objective of this lab is to design an assembler that translates DLX assembly language programs into Intel '.mif' files (one file for data segment and one for code segment). The assembler should be able to handle the entire DLX instruction set discussed in class.

The second objective of this lab is to write a DLX assembly program that computes the factorial of a number using at least one function call. The program must use two scalar variables: `n`, which stores the original input value, and `f`, which stores the resulting factorial.

Procedure

The main program works by reading the input file and processing it in two passes. The program does this by calling the functions `pass_one`, `pass_two_data`, and `pass_two_inst`. Each function is described below:

`pass_one:`

This function is responsible for scanning the assembly source file and gathering all information needed before the machine code can be generated. During this pass, each line is read and cleaned to remove comments, commas, and unnecessary white space. The function then determines which type of category the lines of labels and instructions fall into. These categories are determined using the key words `.data` and `.text` from the DLX file. When in the `.text` section, this function detects labels and records the name and value of that label into `label_table`. The labels are later used for jump and branch instructions (such as `J`, `BNEZ`, etc.). Instructions are stored with their associated addresses, while data declarations are saved separately without final memory addresses. By the end of the first pass, the assembler has built the label symbol table and organized the raw instruction and data lines for later processing.

`pass_two_data:`

This function handles the second-pass processing of the `.data` section by assigning concrete memory addresses and generating the data memory initialization output. The function works by iterating through each stored data declaration, assigning the next available data memory address to that declaration, and recording the address into a data table (`data_table`). The data addresses in `data_table` can then be used later for load and store instructions.

`pass_two_inst:`

This function performs the second-pass processing for the `.text` section. If the line is an instruction line, it is categorized by its instruction field and sent into one of four functions: `parse_register`, `parse_immediate`, `parse_memory`, or `parse_jump`. Each function returns a 32-bit unsigned integer and then prints to the output file.

Aside from the main file, the program is broken up into the four different instruction fields: Register, Immediate, Jump, and Memory. These fields and their functions are described below. The memory allocation shown in Figures 1 - 4 was used to make tables of machine code associated with the opcodes, registers, immediate values, and addresses.

Register

The register instruction logic is implemented in `Register.cpp` and is responsible for converting a register-type DLX assembly instruction into its corresponding 32-bit hexadecimal machine code representation. The function `parse_register` receives a cleaned line of assembly code from the main assembler flow and returns the encoded instruction so it can be written to the instruction memory output file. Internally, `parse_register` calls `parseInstructionLine1`, which breaks up the instruction into four string fields: the opcode, destination register, first source register, and second source register. These string fields and their bit Allocation can be seen in Figure 1.

3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0		
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
opcode				rd				rs1				rs2				unused															

Figure 1: Register Bit Allocation

Once the instruction fields are extracted, the opcode is checked for the NOP instruction, in which case a value of zero is immediately returned. For all other instructions, each field is mapped to its corresponding binary value using predefined lookup tables for opcodes and registers. The final machine instruction is generated by bitwise OR'ing the encoded opcode and register fields together and returning the result.

Table 1 lists all supported DLX register instructions in the register field. This table serves as the reference used by the opcode lookup logic during register instruction encoding.

Table 1: DLX Register Instructions

Mnemonic	Description	Meaning	Opcode (6 bits)
NOP	No Operation	Do nothing	0x00
ADD rd, rs1, rs2	Add (signed)	$rd = rs1 + rs2$	0x03
ADDU rd, rs1, rs2	Add (unsigned)	$rd = rs1 + rs2$	0x05
SUB rd, rs1, rs2	Subtract (signed)	$rd = rs1 - rs2$	0x07
SUBU rd, rs1, rs2	Subtract (unsigned)	$rd = rs1 - rs2$	0x09
AND rd, rs1, rs2	And	$rd = rs1 \wedge rs2$	0x0B
OR rd, rs1, rs2	Or	$rd = rs1 \vee rs2$	0x0D
XOR rd, rs1, rs2	Exclusive Or	$rd = rs1 \oplus rs2$	0x0F
SLL rd, rs1, rs2	Shift Left Logical	$rd = rs1 \ll rs2$	0x11
SRL rd, rs1, rs2	Shift Right Logical	$rd = rs1 \gg rs2$	0x13
SRA rd, rs1, rs2	Shift Right Arithmetic	$rd = rs1 \gg rs2$ (sign)	0x15
SLT rd, rs1, rs2	Set Less Than (signed)	$rd = 1$ if $rs1 < rs2$	0x17
SLTU rd, rs1, rs2	Set Less Than (unsigned)	$rd = 1$ if $rs1 < rs2$	0x19
SGT rd, rs1, rs2	Set Greater Than (signed)	$rd = 1$ if $rs1 > rs2$	0x1B
SGTU rd, rs1, rs2	Set Greater Than (unsigned)	$rd = 1$ if $rs1 > rs2$	0x1D
SLE rd, rs1, rs2	Set Less or Equal (signed)	$rd = 1$ if $rs1 \leq rs2$	0x1F
SLEU rd, rs1, rs2	Set Less or Equal (unsigned)	$rd = 1$ if $rs1 \leq rs2$	0x21
SGE rd, rs1, rs2	Set Greater or Equal (signed)	$rd = 1$ if $rs1 \geq rs2$	0x23
SGEU rd, rs1, rs2	Set Greater or Equal (unsigned)	$rd = 1$ if $rs1 \geq rs2$	0x25
SEQ rd, rs1, rs2	Set Equal	$rd = 1$ if $rs1 = rs2$	0x27
SNE rd, rs1, rs2	Set Not Equal	$rd = 1$ if $rs1 \neq rs2$	0x29

Immediate

Immediate-type instruction handling is implemented in `Immediate.cpp` and is responsible for encoding DLX instructions that include a 16-bit immediate value or label-based operand. The function `parse_immediate` accepts a cleaned assembly instruction line and the label table generated during the first pass of the assembler. The instruction is broken up to extract the opcode, destination register, source register, and an immediate value. The immediate type covers arithmetic operations on immediate values as well as branch instructions. Branch instructions have two operands while typical immediate instructions have three. If no third operand is detected, the instruction is treated as a branch. This means the string that would typically point to a source register name points to a label name instead, and the appropriate address is retrieved from the `label_table`. The opcode and register fields are encoded using predefined lookup tables and combined with the immediate field using bitwise OR operations to produce the final 32-bit machine instruction. The bit allocation for these four string fields is shown in Figure 2.

3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
opcode				rd				rs1				immediate																			

Figure 2: Immediate Bit Allocation

Table 2 lists all supported DLX register instructions in the immediate field. This table serves as the reference used by the opcode lookup logic during immediate instruction encoding.

Table 2: DLX Immediate Instructions

Mnemonic	Description	Meaning	Opcode (6 bits)
ADDI rd, rs1, imm	Add Immediate (signed)	$rd = rs1 + imm$	0x04
ADDUI rd, rs1, imm	Add Immediate (unsigned)	$rd = rs1 + imm$	0x06
SUBI rd, rs1, imm	Subtract Immediate (signed)	$rd = rs1 - imm$	0x08
SUBUI rd, rs1, imm	Subtract Immediate (unsigned)	$rd = rs1 - imm$	0x0A
ANDI rd, rs1, imm	And Immediate	$rd = rs1 \wedge imm$	0x0C
ORI rd, rs1, imm	Or Immediate	$rd = rs1 \vee imm$	0x0E
XORI rd, rs1, imm	Exclusive Or Immediate	$rd = rs1 \oplus imm$	0x10
SLLI rd, rs1, imm	Shift Left Logical Immediate	$rd = rs1 \ll imm$	0x12
SRLI rd, rs1, imm	Shift Right Logical Immediate	$rd = rs1 \gg imm$	0x14
SRAI rd, rs1, imm	Shift Right Arithmetic Immediate	$rd = rs1 \ggg imm$	0x16
SLTI rd, rs1, imm	Set Less Than Immediate	$rd = 1 \text{ if } rs1 < imm$	0x18
SLTUI rd, rs1, imm	Set Less Than Immediate (u)	$rd = 1 \text{ if } rs1 < imm$	0x1A
SGTI rd, rs1, imm	Set Greater Than Immediate	$rd = 1 \text{ if } rs1 > imm$	0x1C
SGTUI rd, rs1, imm	Set Greater Than Immediate (u)	$rd = 1 \text{ if } rs1 > imm$	0x1E
SLEI rd, rs1, imm	Set Less or Equal Immediate	$rd = 1 \text{ if } rs1 \leq imm$	0x20
SLEUI rd, rs1, imm	Set Less or Equal Immediate (u)	$rd = 1 \text{ if } rs1 \leq imm$	0x22
SGEI rd, rs1, imm	Set Greater or Equal Immediate	$rd = 1 \text{ if } rs1 \geq imm$	0x24
SGEUI rd, rs1, imm	Set Greater or Equal Immediate (u)	$rd = 1 \text{ if } rs1 \geq imm$	0x26
SEI rd, rs1, imm	Set Equal Immediate	$rd = 1 \text{ if } rs1 = imm$	0x28
SNEI rd, rs1, imm	Set Not Equal Immediate	$rd = 1 \text{ if } rs1 \neq imm$	0x2A

Jump

Jump-type instruction encoding is handled in `Jump.cpp` and supports control-flow instructions that modify the program counter. The `parse_jump` function parses the opcode and a single secondary parameter, which may represent either a label or a register depending on the instruction type. For absolute jump instructions such as `J` and `JAL`, the secondary parameter is interpreted as a label, and its address is retrieved from the label table created during the first assembler pass. For register-based jump instructions such as `JR` and `JALR`, the secondary parameter is treated as a source register and encoded using the register lookup table. The opcode and address fields or register fields are then combined to form the final machine instruction. The bit allocation for the opcode and address or register is shown in Figure 3.

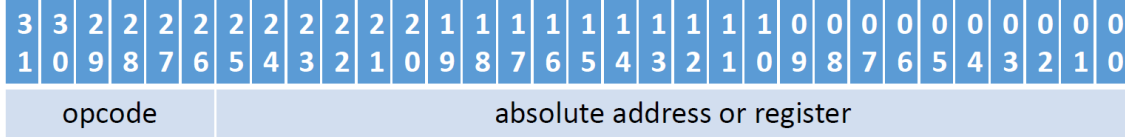


Figure 3: Jump Bit Allocation

Table 3 defines the supported jump operations and their opcode values, serving as a reference for correct instruction encoding.

Table 3: DLX Jump Instructions

Mnemonic	Description	Meaning	Opcode (6 bits)
J label	Jump	pc = label	0x2D
JR rs1	Jump Register	pc = rs1	0x2E
JAL label	Jump and Link	r31 = pc, pc = label	0x2F
JALR rs1	Jump and Link Register	r31 = pc, pc = rs1	0x30

Memory

Memory-type instruction processing is implemented in `Memory.cpp` and is responsible for encoding load and store operations that access data memory. The `parse_memory` function parses the instruction line to determine whether the operation is a load (`LW`) or store (`SW`), as the operand ordering differs between these two instructions. The function extracts the opcode, data register, offset register, and variable name representing the base memory address. The opcode and register operands are encoded using lookup tables, while the base address is resolved by querying the data table constructed during the second data pass of the assembler. These fields are then combined using bitwise OR operations to produce the final 32-bit machine instruction. The bit allocation for the opcode, register data, register offset, and base address is shown in Figure 4 below.

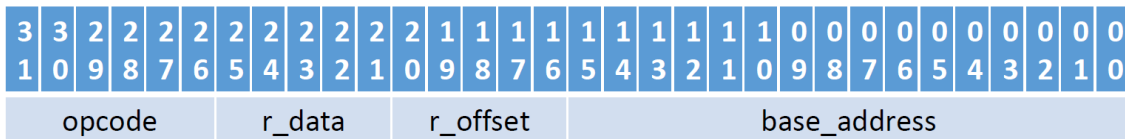


Figure 4: Memory Bit Allocation

Table 4 documents the supported load and store instructions, their meanings, and opcode values. This table serves as the reference used during instruction encoding.

Table 4: DLX Memory Instructions

Mnemonic	Description	Meaning	Opcode (6 bits)
LW r_data, base(r_offset)	Load Word	$r_data = \{base + r_offset\}$	0x01
SW base(r_offset), r_data	Store Word	$\{base + r_offset\} = r_data$	0x02

Factorial

This section addresses the second part of the main objective of this lab which is to write a DLX assembly program that computes the factorial of a number. The program starts by loading the initial factorial value into a register and loading the input value `n` as a loop counter. In each iteration of the outer loop, the program checks whether the counter has reached zero. If the counter has reached zero, the program exits the loop and shows the result (since factorial is $(x - 1) \times (x - 2) \times (x - 3) \times \dots \times (x - n)$). If the counter has not reached zero, the program decrements the counter and calls a `multiply` subroutine using `JAL`.

The `multiply` subroutine simulates multiplication by repeatedly adding a temporary copy of the running total to itself $(x - n)$ times, and then returns the value using `JR`.

The computed factorial value is then stored back into memory at `f`.

Challenges

Throughout the process of designing this program, we faced many challenges. Some of these challenges were with reading the tables designed for making the 32-bit machine code, other challenges we faced dealt with parsing and reading the instruction string lines correctly.

For the opcodes and registers, we wanted to be able to take the piece of the string itself and look up what the op code was. We started by defining macros as strings (for example `#define ADDI "ADDI"`), but then, we realized that that wouldn't work. Instead, we made an unordered map that we could go through and find the opcode or register with its matching 32-bit address. This solved our problem.

We also had another table called mnemonics that looked at our op code and gave us the type it was (register, immediate, memory, etc). We had some issues with the mnemonic table where we weren't able to find the opcode when looking through the table. We realized we were comparing the table to the full instruction line and not just the op code. We fixed it to just compare the op code and it worked.

Results

We were able to take a DLX file and produce two different files: data and code. These files contain machine code. The files we output contain valuable comments that identify what line of code we were given before turning it into a hexadecimal value representing the encoded bits. We also created a DLX assembly file that was able to take a number and find the factorial of it. We also ran the program we wrote and the program turned the DLX input file into data and code files with the required lines in it.

Conclusion

In this lab, we learned about the conversion of DLX assembly instructions into machine code. The lab was challenging enough for us to spend a significant amount of time on it, but not so hard that we couldn't accomplish it. In this assembler lab, we were able to utilize C and C++ and expand our understanding of tables, hash maps, storing strings, and other powerful programming skills.

Appendix A:main.cpp

```

1 #include "stdio.h"
2 #include "line_vec.h"
3 #include <ctype.h>
4 #include <limits.h>
5 #include <stdlib.h>
6 #include <stddef.h> // size_t
7 #include <string.h> // strcmp
8 #include <cstdint>
9 #include <iostream>
10 #include "register.h"
11 #include "immediate.h"
12 #include "memory.h"
13 #include "mothership.h"
14 #include "jump.h"
15 #include "stb_ds.h"
16
17 #define BASE_LINE_LENGTH 256
18 #define INITIAL_ADDR 0
19
20 void strip(char *s)
21 {
22     char *src = s;
23     char *dst = s;
24
25     int prev_char = 0; // prevents leading spaces
26
27     while (*src) {
28         unsigned char c = (char)*src++;
29
30         if (c == ';' ) break; // comment starts: ignore rest of line
31
32         if (isspace(c)) {
33             if (!prev_char)
34             {
35                 continue;
36             } else
37             {
38                 *dst++ = ' ';
39                 prev_char = 0;
40                 continue;
41             }
42         }
43
44         if (c == ',' || c=='')
45         {
46             continue;
47         }
48         if (c=='(')
49         {
50             *dst++ = ' ';
51             prev_char = 0;
52             continue;
53         }
54
55         // Keep everything else
56         *dst++ = (char)c;
57         prev_char = 1;
58     }
59
60     // remove trailing space if present
61     if (dst > s && dst[-1] == ' ')
62         dst--;
63
64     *dst = '\0';
65 }

```

```

66
67 char* try_parse_label(char* line, mnemonic_entry* mnemonic_table)
68 {
69     char* save = NULL;
70     char* label = strtok_r(strdup(line), " ", &save);
71     if (shgeti(mnemonic_table, label) == -1)
72     {
73
74         return (label);
75     }
76     return(NULL);
77 }
78
79 void pass_one(FILE* infile, Label** label_table, line_vec* data_lines, line_vec* text_lines,
    mnemonic_entry* mnemonic_table)
80 {
81     char* buf = (char*)calloc(1, BASE_LINE_LENGTH);
82     linevec_init(data_lines);
83     linevec_init(text_lines);
84     uint32_t pc = 0;
85     int label_idx = 0;
86     int line_num = -1;
87     Section mode = SEC_NONE;
88
89     while(fgets(buf, BASE_LINE_LENGTH, infile))
90     {
91         char* line = buf;
92         line_num++;
93         strip(line); // Strips all commas and most whitespace from
94                     // each line (leaves a single space between each word)
95         if (!*line){continue;}
96
97         if (strcmp(line, ".data") == 0)
98         {
99             mode = SEC_DATA;
100            continue;
101        } else if (strcmp(line, ".text") == 0)
102        {
103            mode = SEC_TEXT;
104            continue;
105        }
106
107        if (mode == SEC_TEXT)
108        {
109            char* label_name = try_parse_label(line, mnemonic_table); // looks for a label
110            if (label_name != NULL)
111            {
112                shput(*label_table, strdup(label_name), pc);
113                continue;
114            }
115            linevec_push(text_lines, line_num, pc++, strdup(line));
116        } else if (mode == SEC_DATA)
117        {
118            linevec_push(data_lines, line_num, INITIAL_ADDR, strdup(line)); // Push The
119            data line to the record.
120
121            // All addresses will initialize
122            to 0, that will be corrected on the second pass.
123        }
124        free(buf);
125    }
126 }
127
128 void pass_two_inst(FILE* out_code, Label** data_table, Label** label_table, mnemonic_entry*
    mnemonic_table, line_vec* text_lines)
129 {
130     // Instruction pass two
131     printf("Number of instruction lines: %d\n", text_lines->len);
132     for (int i = 0; i < text_lines->len; i++)

```

```

130 {
131     char* save=NULL;
132     printf("%s\n", text_lines->items[i].text);
133     char* line = strdup(text_lines->items[i].text);
134     char* op_word = strtok_r(line, " ", &save);
135     mnemonic_entry* entry = shgetp_null(mnemonic_table, op_word);
136     if (entry == NULL)
137     {
138         std::cerr << "Instruction not found. " << op_word << std::endl;
139         abort();
140     }
141     inst_type type = entry->value;
142     uint32_t instruction=0;
143     switch (type)
144     {
145         case IMMEDIATE:
146             //printf("Immediate type found.\n");
147             instruction = parse_immediate(text_lines->items[i].text, label_table);
148             break;
149         case MEMORY:
150             //printf("Memory type found.\n");
151             instruction = parse_memory(text_lines->items[i].text, data_table);
152             break;
153         case REGISTER:
154             //printf("Register type found.\n");
155             instruction = parse_register(text_lines->items[i].text);
156             break;
157         case JUMP:
158             //printf("Jump type found.\n");
159             instruction = parse_jump(text_lines->items[i].text, label_table);
160             break;
161         default:
162             //printf("Invalid type type found.\n");
163             instruction = 0; //NOP if we somehow get here.
164             break;
165     }
166     fprintf(out_code, "%03X : %08X; --%s\n", text_lines->items[i].addr, instruction,
167         text_lines->items[i].text);
168     free(line);
169 }
170 }
171
172 void pass_two_data(FILE* out_data, line_vec* data_lines, Label** data_table)
173 {
174     uint16_t data_addr = 0;
175     for (int i = 0; i < data_lines->len; i++)
176     {
177         char* save = NULL;
178         data_lines->items[i].addr = data_addr;
179         line_rec curr_line = data_lines->items[i];
180         char* line = curr_line.text;
181         char* var_name = strdup(strtok_r(line, " ", &save));
182         shput(*data_table, var_name, data_addr);
183         // "m 10 0 4 0 5 6..."
184         // Format for strtol(): long val = strtol(s, &end, 10);
185         char* num_str = strtok_r(NULL, " ", &save); // get size as a str
186         int size = (int)strtol(num_str, NULL, 10); // convert size to integer
187         uint32_t data;
188         for (int j = 0; j < size; j++)
189         {
190             num_str = strtok_r(NULL, " ", &save); //
191             data = (uint32_t)strtol(num_str, NULL, 10);
192             fprintf(out_data, "%03X : %08X; --s[%d]\n", data_addr+j, data, var_name, j);
193         }
194         data_addr += size;
195     }
196     fprintf(out_data, "\nEND;\n");

```



```

197 }
198
199 void build_mnemonic_table(mnemonic_entry** table_ptr);
200
201 int main(int argc, char* argv[])
202 {
203     // if (argc != 4)
204     // {
205     //     printf("not enough files");
206     //     return 0;
207     // }
208     //defining the input file
209     FILE *input_file= fopen(argv[1], "r");
210     //defining and opening the output files
211     FILE *out_data= fopen(argv[2], "w");
212     FILE *out_code= fopen(argv[3], "w");
213
214     fprintf(out_code, "DEPTH = 1024;\n"
215                     "WIDTH = 32;\n"
216                     "ADDRESS_RADIX = HEX;\n"
217                     "DATA_RADIX = HEX;\n"
218                     "CONTENT\n"
219                     "BEGIN\n\n");
220     fprintf(out_data, "DEPTH = 1024;\n"
221                     "WIDTH = 32;\n"
222                     "ADDRESS_RADIX = HEX;\n"
223                     "DATA_RADIX = HEX;\n"
224                     "CONTENT\n"
225                     "BEGIN\n\n");
226
227     Label* label_table = NULL;
228     mnemonic_entry* mnemonic_table = NULL;
229     build_mnemonic_table(&mnemonic_table);
230
231     int len=shlen(mnemonic_table);
232     for(int i =0; i<len;i++)
233     {
234         printf("%s\t%d\n", mnemonic_table[i].key, mnemonic_table[i].value);
235     }
236
237     line_vec data_lines, text_lines;
238     pass_one(input_file, &label_table, &data_lines, &text_lines, mnemonic_table);
239     Label* data_table = NULL;
240     pass_two_data(out_data, &data_lines, &data_table);
241     printf("Finished data pass two, on to instruction pass two...\n");
242     pass_two_inst(out_code, &data_table, &label_table, mnemonic_table, &text_lines);
243
244     fclose(out_data);
245     fclose(out_code);
246
247     shfree(mnemonic_table);
248     shfree(label_table);
249     shfree(data_table);
250     return 0;
251 }
252
253 void build_mnemonic_table(mnemonic_entry** table_ptr)
254 {
255     mnemonic_entry* table = NULL;
256     shput(table, "ADD", REGISTER);
257     shput(table, "ADDI", IMMEDIATE);
258     shput(table, "ADDU", REGISTER);
259     shput(table, "ADDUI", IMMEDIATE);
260
261     shput(table, "AND", REGISTER);
262     shput(table, "ANDI", IMMEDIATE);
263
264     shput(table, "BEQZ", IMMEDIATE);

```

```

265     shput(table, "BNEZ", IMMEDIATE);
266
267     shput(table, "J", JUMP);
268     shput(table, "JAL", JUMP);
269     shput(table, "JALR", JUMP);
270     shput(table, "JR", JUMP);
271
272     shput(table, "LW", MEMORY);
273
274     shput(table, "NOP", REGISTER);
275
276     shput(table, "OR", REGISTER);
277     shput(table, "ORI", IMMEDIATE);
278
279     shput(table, "SEQ", REGISTER);
280     shput(table, "SEI", IMMEDIATE);
281
282     shput(table, "SGE", REGISTER);
283     shput(table, "SGEI", IMMEDIATE);
284     shput(table, "SGEU", REGISTER);
285     shput(table, "SGEUI", IMMEDIATE);
286
287     shput(table, "SGT", REGISTER);
288     shput(table, "SGTI", IMMEDIATE);
289     shput(table, "SGTU", REGISTER);
290     shput(table, "SGTUI", IMMEDIATE);
291
292     shput(table, "SLE", REGISTER);
293     shput(table, "SLEI", IMMEDIATE);
294     shput(table, "SLEU", REGISTER);
295     shput(table, "SLEUI", IMMEDIATE);
296
297     shput(table, "SLL", REGISTER);
298     shput(table, "SLLI", IMMEDIATE);
299
300     shput(table, "SLT", REGISTER);
301     shput(table, "SLTI", IMMEDIATE);
302     shput(table, "SLTU", REGISTER);
303     shput(table, "SLTUI", IMMEDIATE);
304
305     shput(table, "SNE", REGISTER);
306     shput(table, "SNEI", IMMEDIATE);
307
308     shput(table, "SRA", REGISTER);
309     shput(table, "SRAI", IMMEDIATE);
310
311     shput(table, "SRL", REGISTER);
312     shput(table, "SRLI", IMMEDIATE);
313
314     shput(table, "SUB", REGISTER);
315     shput(table, "SUBI", IMMEDIATE);
316     shput(table, "SUBU", REGISTER);
317     shput(table, "SUBUI", IMMEDIATE);
318
319     shput(table, "SW", MEMORY);
320
321     shput(table, "XOR", REGISTER);
322     shput(table, "XORI", IMMEDIATE);
323     printf("Table Built!\n");
324     *table_ptr=table;
325 }
326
327 // int main()
328 // {
329 //     char str[] = "\tADDI R1, R0, 0    ;skurp";
330 //     strip(str);
331 //     printf("%s\n", str);
332

```

```
333 //      return(0);
334 // }
335
336 // int main()
337 // {
338 //      char line1[] = "LW R10 n R0";
339 //      char line2[] = "SW result R0 R3";
340
341 //      Label* data_table = NULL;
342 //      shput(data_table, "n", 0);
343 //      shput(data_table, "result", 1);
344
345 //      uint32_t inst = parse_memory(line1, data_table);
346 //      printf("first instruction:\t\t%08X\n", inst);
347
348 //      inst = parse_memory(line2, data_table);
349 //      printf("second instruction:\t\t%08X\n", inst);
350
351 //      shfree(data_table);
352 //      return(0);
353 // }
```

Appendix B: register.cpp

```

1 #include <iostream>
2 #include <sstream>
3 #include <string>
4 #include <algorithm> // required for std::
5 #include <cstdint>
6 #include "mothership.h"
7 #include "register.h"
8
9 using namespace std;
10
11
12 Instruction_Register parseInstructionLine1(const string& line)
13 {
14     Instruction_Register inst;
15     string cleaned = line;
16
17     cleaned.erase(remove(cleaned.begin(), cleaned.end(), ','), cleaned.end());
18
19     istringstream iss(cleaned);
20     iss >> inst.opcode >> inst.regOut >> inst.regPri >> inst.regSec;
21
22     return inst;
23 }
24 // Added a pointer to the beginning of the line. -Jackson
25 uint32_t parse_register(char* line_ptr)
26 {
27     uint32_t opcode, regOut, regPri, regSec, finalresult;
28     string line = line_ptr; // This is how you convert from C style strings
29                             // to C++ style strings. -Jackson
30
31     Instruction_Register inst = parseInstructionLine1(line);
32
33
34     auto it = opcodeTable.find(inst.opcode);
35     opcode = it->second;
36     if (inst.opcode == "NOP")
37     {
38         return 0;
39     }
40
41     it = destRegTable.find(inst.regOut);
42     regOut = it->second;
43
44     it = rs1Table.find(inst.regPri);
45     regPri = it->second;
46
47     it = rs2Table.find(inst.regSec);
48     regSec = it->second;
49
50
51     //OR them together
52     finalresult = opcode | regOut | regPri | regSec;
53     cout << "0x" << hex << finalresult << endl;
54
55
56 /* just printing out the line piece by piece
57    for (const auto& inst : instructions) {
58        cout << opcode << " "
59            << inst.regOut << " "
60            << inst.regPri << " "
61            << inst.regSec << endl;
62    }
63 */
64     return finalresult;
65 }

```

Appendix C: immediate.cpp

```

1 #include <iostream>
2 #include <sstream>
3 #include <string>
4
5 #include "stb_ds.h"
6 #include "immediate.h"
7
8 #include "mothership.h"
9
10
11 using namespace std;
12
13 uint32_t parse_immediate(char* line_ptr, Label** label_table)
14 {
15     string line = line_ptr;
16     uint32_t opcodeBits, regOutBits, regPriBits, immBits, machineCode=0;
17
18     Instruction_Imm inst;
19     istringstream iss(line);
20     iss >> inst.opcode >> inst.regOut >> inst.regPri;
21
22     // 4) immediate
23     if (!(iss >> inst.immStr)) // Must be a branch.
24     {
25         immBits = (shget(*label_table, inst.regPri.c_str())) & 0x0000FFFF;
26         regPriBits = 0;
27     } else
28     {
29
30         immBits = static_cast<uint32_t>(stoi(inst.immStr));
31
32         //3) rs1 (primary)
33         auto rs1It = rs1Table.find(inst.regPri);
34         if (rs1It == rs1Table.end())
35         {
36             cerr << "Unknown primary register: " << inst.regPri << endl;
37             abort();
38         }
39
40         regPriBits = rs1It->second;
41     }
42
43     // 1) opcode
44     auto opIt = opcodeTable.find(inst.opcode);
45     if (opIt == opcodeTable.end())
46     {
47         cerr << "Unknown opcode: " << inst.opcode << endl;
48         abort();
49     }
50
51     opcodeBits = opIt->second;
52
53     // 2) rd (dest)
54     auto rdIt = destRegTable.find(inst.regOut);
55     if (rdIt == destRegTable.end())
56     {
57         cerr << "Unknown opcode: " << inst.regOut << endl;
58         abort();
59     }
60
61     regOutBits = rdIt->second;
62
63
64
65

```

```
66 // 5) build final instruction word
67 // ASSUMPTION: immediate is bits [15:0]
68 machineCode = opcodeBits | regOutBits | regPriBits | immBits;
69
70 return machineCode;
71 }
```

Appendix D: jump.cpp

```

1 #include <iostream>
2 #include <sstream>
3 #include <string>
4
5 #include <cstdint>
6 #include "stb_ds.h"
7 #include "mothership.h"
8 #include "jump.h"
9
10 using namespace std;
11
12 uint32_t parse_jump(char* line_ptr, Label** label_table)
13 {
14     string line = line_ptr;
15     uint32_t opcodeBits, addrBits, machineCode=0;
16
17     istringstream iss(line);
18     Instruction_Jump inst;
19     iss >> inst.opcode >> inst.sec_param;
20
21     // 1) opcode
22     auto opIt = opcodeTable.find(inst.opcode);
23     if (opIt == opcodeTable.end())
24     {
25         cerr << "Unknown opcode: " << inst.opcode << endl;
26         abort();
27     }
28
29     opcodeBits = opIt->second;
30
31     // 2)
32     if (inst.opcode == "J" || inst.opcode == "JAL")
33     {
34         addrBits = shget(*label_table, inst.sec_param.c_str());
35     }
36     else if (inst.opcode == "JR" || inst.opcode == "JALR")
37     {
38         auto rs1It = rs1Table.find(inst.sec_param);
39         if (rs1It == rs1Table.end())
40         {
41             cerr << "Unknown primary register: " << inst.sec_param << endl;
42             abort();
43         }
44
45         addrBits = (rs1It->second) >> 16;
46     }
47
48     // 5) build final instruction word
49     // ASSUMPTION: immediate is bits [15:0]
50     machineCode = opcodeBits | addrBits;
51     return machineCode;
52 }

```

Appendix E:memory.cpp

```

1 #include <iostream>
2 #include <sstream>
3 #include <string>
4
5 #include "stb_ds.h"
6 #include "memory.h"
7 #include "mothership.h"
8
9
10 using namespace std;
11
12     // char line1[] = "LW R10 n R0";
13     // char line2[] = "SW result R1 R12";
14
15 uint32_t parse_memory(char* line_ptr, Label** data_table)
16 {
17     string line = line_ptr;
18     uint32_t opcodeBits, rDataBits, rOffsetBits, baseAddrBits, machineCode=0;
19
20     Instruction_Memory inst;
21     istringstream iss(line);
22     iss >> inst.opcode;
23     if (inst.opcode == "SW"){
24         iss >> inst.varName >> inst.rOffset >> inst.rData;
25     } else if (inst.opcode == "LW")
26     {
27         iss >> inst.rData >> inst.varName >> inst.rOffset;
28     }
29
30     // 1) opcode
31     auto opIt = opcodeTable.find(inst.opcode);
32     if (opIt == opcodeTable.end())
33     {
34         cerr << "Unknown opcode: " << inst.opcode << endl;
35         abort();
36     }
37
38     opcodeBits = opIt->second;
39
40     // 2) r_data
41     auto rdIt = destRegTable.find(inst.rData);
42     if (rdIt == destRegTable.end())
43     {
44         cerr << "Unknown opcode: " << inst.rData << endl;
45         abort();
46     }
47
48     rDataBits = rdIt->second;
49
50     //3) r_offset
51     auto rs1It = rs1Table.find(inst.rOffset);
52     if (rs1It == rs1Table.end())
53     {
54         cerr << "Unknown primary register: " << inst.rOffset << endl;
55         abort();
56     }
57
58     rOffsetBits = rs1It->second;
59
60     // 4) Base Address
61
62     baseAddrBits = shget(*data_table, inst.varName.c_str());
63
64
65     // 5) build final instruction word

```



```
66 // ASSUMPTION: immediate is bits [15:0]
67 machineCode = opcodeBits | rDataBits | rOffsetBits | baseAddrBits;
68
69 return machineCode;
70 }
```

Appendix F: Table with 32-bit Binary Values

```

1 //include function calls of files we will use in the main file
2 #ifndef MOTHERSHIP_H
3 #define MOTHERSHIP_H
4
5 #include "stdio.h"
6 #include <unordered_map>
7 #include <string>
8 #include <cstdint>
9
10 typedef enum {IMMEDIATE, MEMORY, REGISTER, JUMP, INVALID} inst_type;
11
12 typedef struct
13 {
14     const char* key;
15     inst_type value;
16 } mnemonic_entry;
17
18 typedef struct {
19     char* key;
20     uint32_t value;
21 } Label;
22
23 const std::unordered_map<std::string, uint32_t> opcodeTable = {
24     {"NOP", 0x00000000},
25
26     {"LW", 0x04000000},
27     {"SW", 0x08000000},
28
29     {"ADD", 0x0C000000},
30     {"ADDI", 0x10000000},
31     {"ADDU", 0x14000000},
32     {"ADDUI", 0x18000000},
33
34     {"SUB", 0x1C000000},
35     {"SUBI", 0x20000000},
36     {"SUBU", 0x24000000},
37     {"SUBUI", 0x28000000},
38
39     {"AND", 0x2C000000},
40     {"ANDI", 0x30000000},
41
42     {"OR", 0x34000000},
43     {"ORI", 0x38000000},
44
45     {"XOR", 0x3C000000},
46     {"XORI", 0x40000000},
47
48     {"SLL", 0x44000000},
49     {"SLLI", 0x48000000},
50     {"SRL", 0x4C000000},
51     {"SRLI", 0x50000000},
52     {"SRA", 0x54000000},
53     {"SRAI", 0x58000000},
54
55     {"SLT", 0x5C000000},
56     {"SLTI", 0x60000000},
57     {"SLTU", 0x64000000},
58     {"SLTUI", 0x68000000},
59
60     {"SGT", 0x6C000000},
61     {"SGTI", 0x70000000},
62     {"SGTU", 0x74000000},
63     {"SGTUI", 0x78000000},
64
65     {"SLE", 0x7C000000},

```

```

66     {"SLEI", 0x80000000},
67     {"SLEU", 0x84000000},
68     {"SLEUI", 0x88000000},
69
70     {"SGE", 0x8C000000},
71     {"SGEI", 0x90000000},
72     {"SGEU", 0x94000000},
73     {"SGEUI", 0x98000000},
74
75     {"SEQ", 0x9C000000},
76     {"SEQI", 0xA0000000},
77     {"SNE", 0xA4000000},
78     {"SNEI", 0xA8000000},
79
80     {"BEQZ", 0xAC000000},
81     {"BNEZ", 0xB0000000},
82
83     {"J", 0xB4000000},
84     {"JR", 0xB8000000},
85     {"JAL", 0xBC000000},
86     {"JALR", 0xC0000000}
87 };
88
89 //destination registers
90
91 const std::unordered_map<std::string, uint32_t> destRegTable = {
92     {"R0", 0x00000000},
93     {"R1", 0x00200000},
94     {"R2", 0x00400000},
95     {"R3", 0x00600000},
96     {"R4", 0x00800000},
97     {"R5", 0x00A00000},
98     {"R6", 0x00C00000},
99     {"R7", 0x00E00000},
100    {"R8", 0x01000000},
101    {"R9", 0x01200000},
102    {"R10", 0x01400000},
103    {"R11", 0x01600000},
104    {"R12", 0x01800000},
105    {"R13", 0x01A00000},
106    {"R14", 0x01C00000},
107    {"R15", 0x01E00000},
108    {"R16", 0x02000000},
109    {"R17", 0x02200000},
110    {"R18", 0x02400000},
111    {"R19", 0x02600000},
112    {"R20", 0x02800000},
113    {"R21", 0x02A00000},
114    {"R22", 0x02C00000},
115    {"R23", 0x02E00000},
116    {"R24", 0x03000000},
117    {"R25", 0x03200000},
118    {"R26", 0x03400000},
119    {"R27", 0x03600000},
120    {"R28", 0x03800000},
121    {"R29", 0x03A00000},
122    {"R30", 0x03C00000},
123    {"R31", 0x03E00000}
124 };
125
126
127
128 const std::unordered_map<std::string, uint32_t> rs1Table = {
129     {"R0", 0x00000000},
130     {"R1", 0x00010000},
131     {"R2", 0x00020000},
132     {"R3", 0x00030000},
133     {"R4", 0x00040000},

```

```

134     {"R5", 0x00050000},
135     {"R6", 0x00060000},
136     {"R7", 0x00070000},
137     {"R8", 0x00080000},
138     {"R9", 0x00090000},
139     {"R10", 0x000A0000},
140     {"R11", 0x000B0000},
141     {"R12", 0x000C0000},
142     {"R13", 0x000D0000},
143     {"R14", 0x000E0000},
144     {"R15", 0x000F0000},
145     {"R16", 0x00100000},
146     {"R17", 0x00110000},
147     {"R18", 0x00120000},
148     {"R19", 0x00130000},
149     {"R20", 0x00140000},
150     {"R21", 0x00150000},
151     {"R22", 0x00160000},
152     {"R23", 0x00170000},
153     {"R24", 0x00180000},
154     {"R25", 0x00190000},
155     {"R26", 0x001A0000},
156     {"R27", 0x001B0000},
157     {"R28", 0x001C0000},
158     {"R29", 0x001D0000},
159     {"R30", 0x001E0000},
160     {"R31", 0x001F0000}
161 };
162
163 const std::unordered_map<std::string, uint32_t> rs2Table = {
164     {"R0", 0x00000000},
165     {"R1", 0x00000F00},
166     {"R2", 0x00001000},
167     {"R3", 0x00001100},
168     {"R4", 0x00001200},
169     {"R5", 0x00001300},
170     {"R6", 0x00001400},
171     {"R7", 0x00001500},
172     {"R8", 0x00001600},
173     {"R9", 0x00001700},
174     {"R10", 0x00001800},
175     {"R11", 0x00001900},
176     {"R12", 0x00001A00},
177     {"R13", 0x00001B00},
178     {"R14", 0x00001C00},
179     {"R15", 0x00001D00},
180     {"R16", 0x00001E00},
181     {"R17", 0x00001F00},
182     {"R18", 0x00002000},
183     {"R19", 0x00002100},
184     {"R20", 0x00002200},
185     {"R21", 0x00002300},
186     {"R22", 0x00002400},
187     {"R23", 0x00002500},
188     {"R24", 0x00002600},
189     {"R25", 0x00002700},
190     {"R26", 0x00002800},
191     {"R27", 0x00002900},
192     {"R28", 0x00002A00},
193     {"R29", 0x00002B00},
194     {"R30", 0x00002C00},
195     {"R31", 0x00002D00}
196 };
197
198 #endif

```